NAVIGÁCIA REACT NATIVE

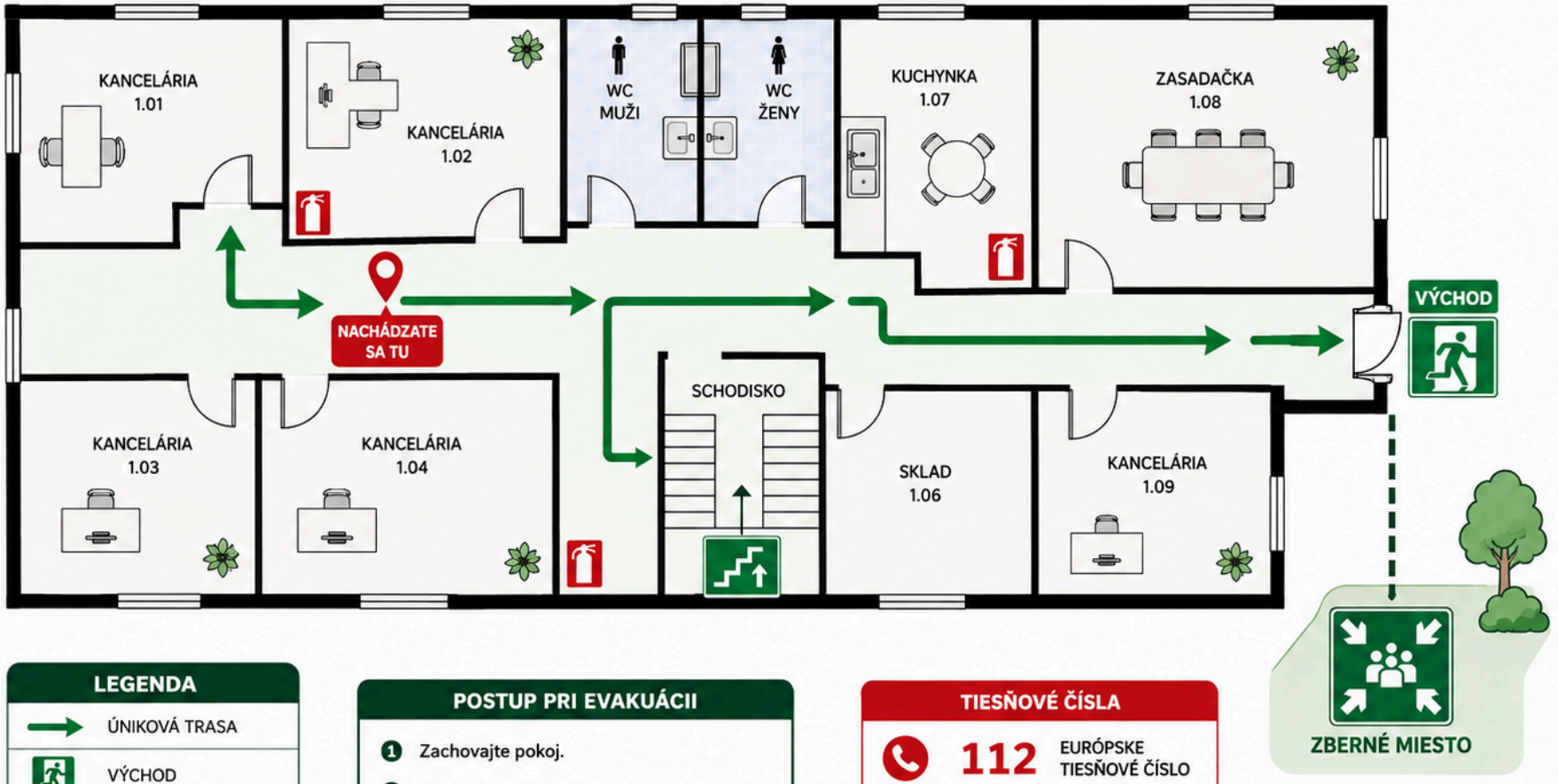
ČO JE TO NAVIGÁCIA?

Vo všeobecnosti znamená určovanie smeru a vedenie z jedného miesta alebo stavu k cieľu.

Môže ísť napríklad o:

POHYB V PRIESTORE
NAVIGÁCIA AUTOM, LOĎOU ČI PEŠO

Navigácia – evakuačný plán



LEGENDA

- ÚNIKOVÁ TRASA
- VÝCHOD
- SCHODISKO
- HASIACI PRÍSTROJ
- ZBERNÉ MIESTO
- NACHÁDZATE SA TU

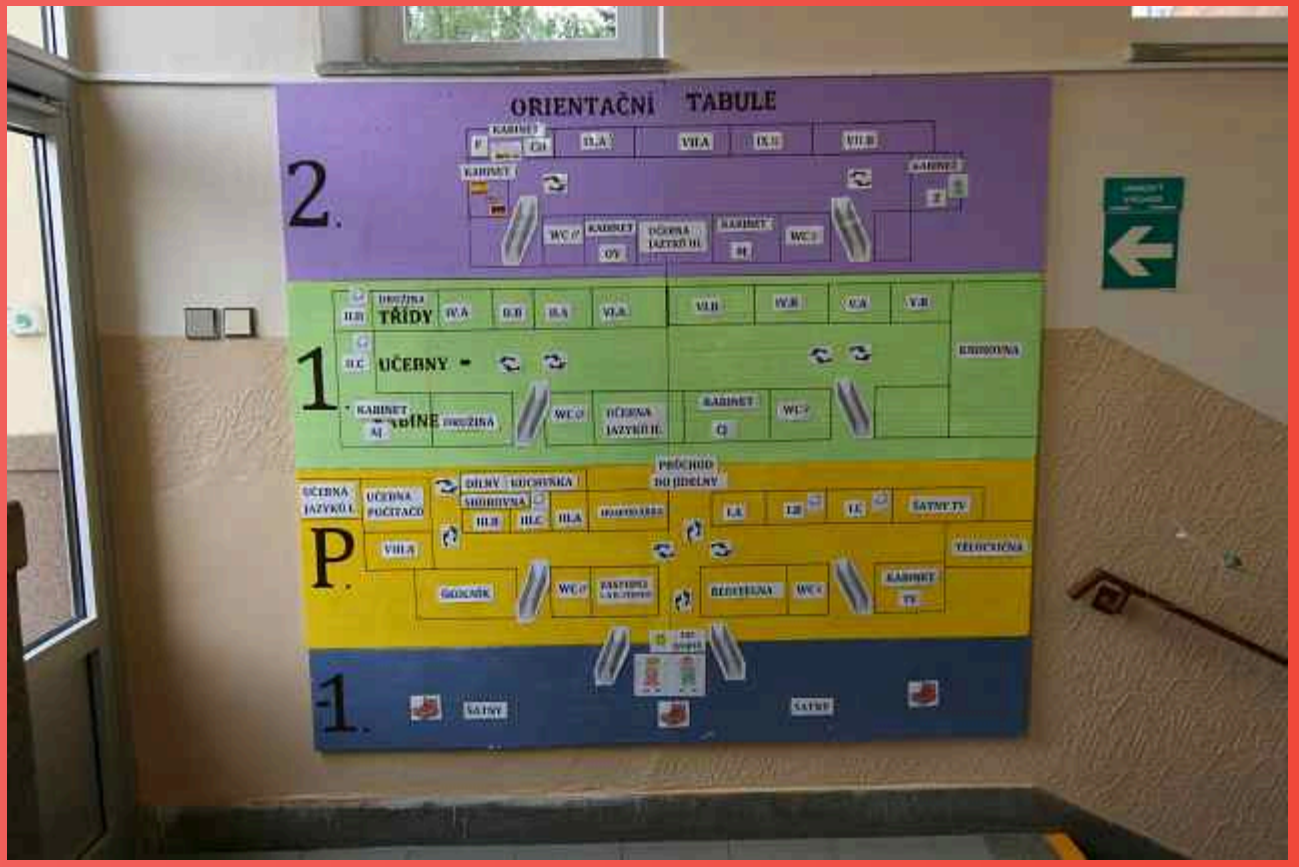
POSTUP PRI EVAKUÁCII

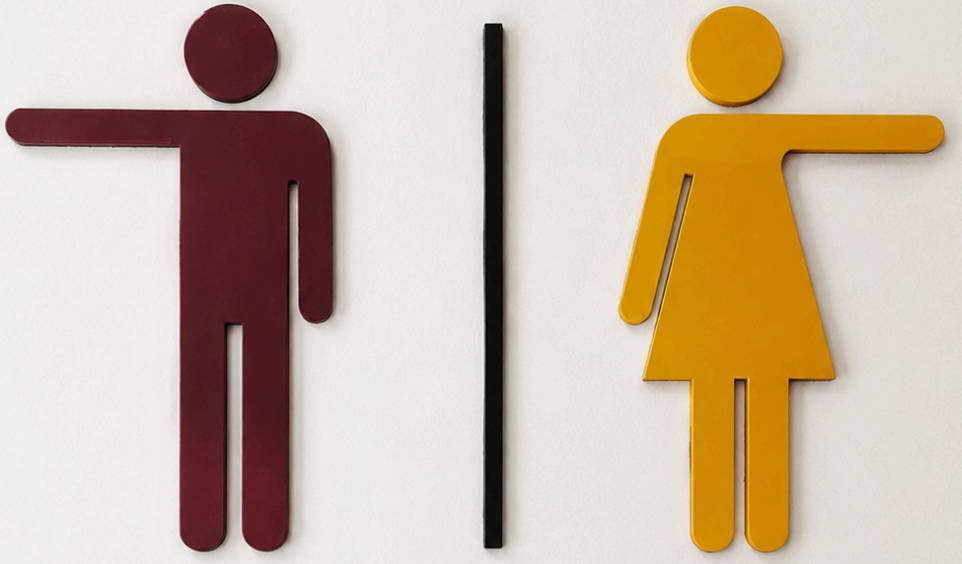
1. Zachovajte pokoj.
2. Okamžite ukončite činnosť.
3. Vydajte sa po vyznačenej únikovej trase najbližším východom.
4. Nepoužívajte výťah.
5. Zhromaždite sa na určenom zbernom mieste.
6. Poskytnite pomoc ostatným, ak je to možné.

TIESŇOVÉ ČÍSLA

112	EURÓPSKE TIESŇOVÉ ČÍSLO
150	HASIČI
155	ZÁCHRANNÁ ZDRAVOTNÁ SLUŽBA
158	POLÍCIA


2. POSCHODIE
STOMATOLOGICKÁ AMBULANCIA
MUDr. Vlasta ŠMONDRKOVÁ
STOMATOLOGICKÁ AMBULANCIA
MUDr. Jana PRUTKAYOVÁ
1. POSCHODIE
VŠEOBECNÁ ABMULANCIA
PRE DOSPELÝCH
MUDr. Peter FLOROVIČ
VŠEOBECNÁ ABMULANCIA
PRE DOSPELÝCH
MUDr. Ľubica ŠUHAJDOVÁ
VŠEOBECNÁ ABMULANCIA
PRE DOSPELÝCH
MUDr. Oľga GRAUSOVÁ
REHABILITAČNÁ AMBULANCIA
MUDr. Jozef ŠIMKO
PRÍZEMIE
GYNEKOLOGICKÁ AMBULANCIA
MUDr. Dušan NIŽNÍK
VŠEOBECNÁ ABMULANCIA
PRE DETI A DORAST
MUDr. Alžbeta LEHOTSKÁ
VŠEOBECNÁ ABMULANCIA
PRE DETI A DORAST
MUDr. Renata HRNČIAROVÁ


ORIENTAČNÍ TABULE
2.
1
TŘÍDY
UČEBNY
P.
-1.

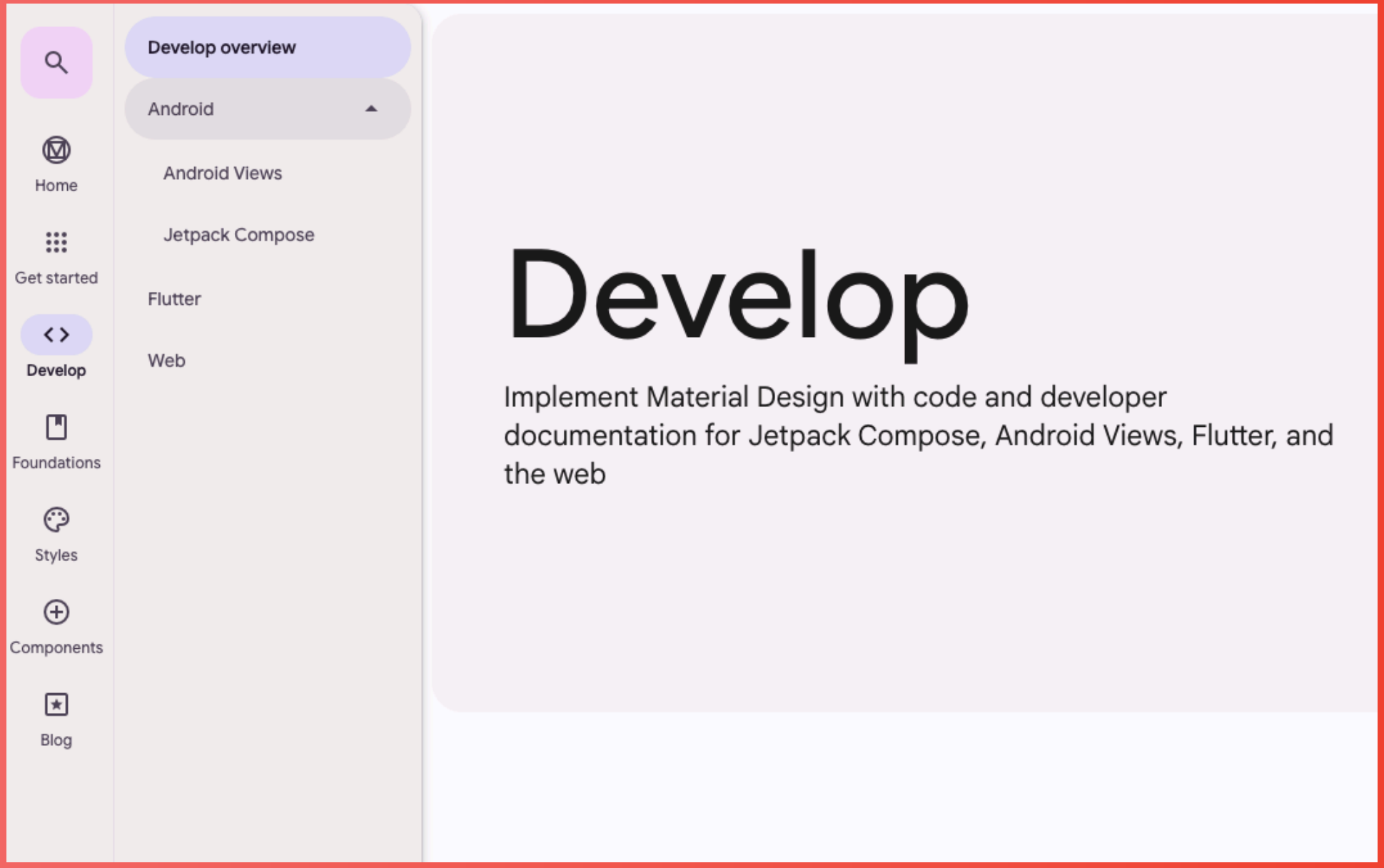


MEN
to the left
because

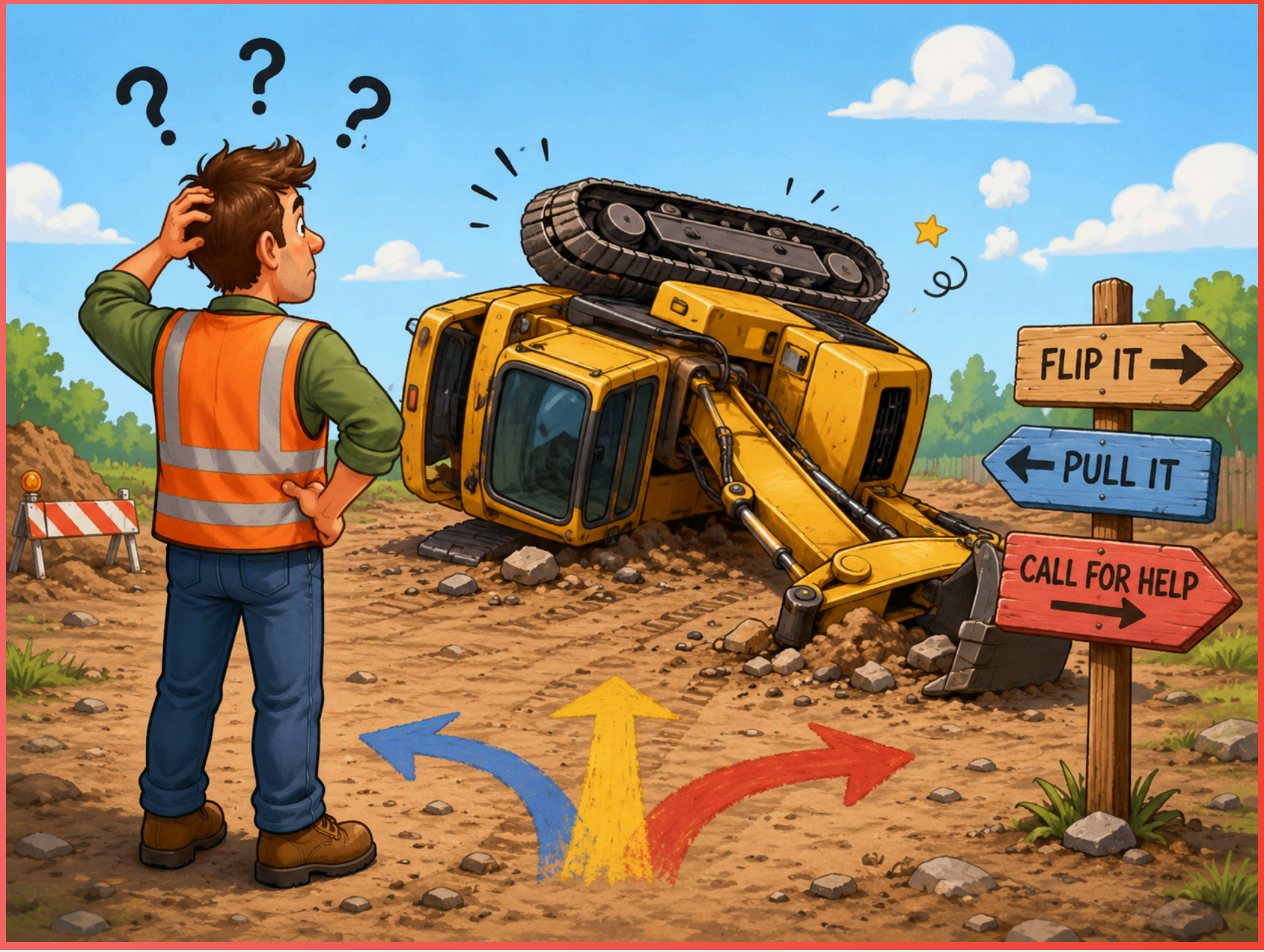
WOMEN
are always
right!



ORIENTÁCIU V APLIKÁCII ALEBO NA WEBE – MENU, ODKAZY A TLAČIDLÁ, KTORÉ ŤA VEDÚ K OBSAHU


Develop overview
Android
Android Views
Jetpack Compose
Flutter
Web
Home
Get started
Develop
Foundations
Styles
Components
Blog
Develop
Implement Material Design with code and developer documentation for Jetpack Compose, Android Views, Flutter, and the web

PRENESENÝ VÝZNAM
„NAVIGOVAŤ SA V ZLOŽITEJ SITUÁCII“,
TEDA VEDIEŤ SA ZORIENTOVAŤ A
ROZHODNÚŤ, AKO POSTUPOVAŤ.


FLIP IT
PULL IT
CALL FOR HELP

HOTOVÉ RIEŠENIA V REACT NATIVE

- React Navigation
- React Native Navigation (Wix)
- Expo Router

REACT NAVIGATION KNIŽNICA

Je navigačná knižnica pre React Native a web. Zabezpečuje logiku smerovania (routing) a UI bežné v mobilných aplikáciách a PWA, s typovo bezpečnou navigáciou, vstavanou podporou univerzálnych odkazov (Deep linking) na mobile a integráciou histórie prehliadača na webe.

TYPY NAVIGÁCIE

- Stack navigácia
- Tab navigácia
- Drawer navigácia

Každý typ navigácie má v React Navigation svoju vlastnú knižnicu, ktorú tiež môžeme označiť ako "navigator":

- pre stack: `@react-navigation/native-stack`
  (alebo `@react-navigation/stack`)
- pre taby: `@react-navigation/bottom-tabs`
  (alebo `@react-navigation/material-top-tabs`)
- pre drawer: `@react-navigation/drawer`

PRAKTICKÁ ČASŤ

`NavigationContainer`

`NavigationContainer` je koreňový komponent z `@react-navigation/native`.

- **Spravuje navigačný stav** – aktuálnu obrazovku a históriu navigácie.
- Obaľuje hlavný navigátor a zabezpečuje podporu deep linkov cez prop `linking`.
- V bežnej aplikácii používame jeden `NavigationContainer`, spoločný pre všetky navigátory.

`navigation` objekt

Objekt `navigation` je „diaľkové ovládanie“ navigácie.

- Poskytuje metódy, ktorými **meníme navigačný stav** – ktoré obrazovky sú v stacku a ktorá je práve zobrazená.
- Každá obrazovka registrovaná v `Stack.Screen` ho dostáva automaticky cez props.

`useNavigation` VS. PROPS

Existujú dva spôsoby, ako sa k nemu dostať, a líšia sa hlavne typovaním:

	props (`{ navigation }`)	`useNavigation()`
Kde je dostupný	iba priamo v komponente obrazovky	v ľubovoľnom vnorenom komponente
Typovanie	**typovaný** cez `NativeStackScreenProps<NavigationParamList, 'ProductList'>` – TS pozná názvy obrazoviek aj ich parametre	**netypovaný** (defaultne `any`-like), typ si musíme doplniť ručne cez generiku
Odporúčanie	preferovaný spôsob v komponente obrazovky	pre hlboko vnorené komponenty (napr. tlačidlo v `ProductCard`), aby sme `navigation` nemuseli posielať cez props "drilling"

```tsx
// typovane — TS skontroluje nazov obrazovky aj parametre
type Props = NativeStackScreenProps<NavigationParamList, 'ProductList'>;

function ProductListScreen({ navigation, route }: Props) {
    navigation.navigate('ProductDetail', { productId: '1' }); // ✅ kontrola typ
}
```

```tsx
const navigation = useNavigation();

// s generikom — dotypovanie, aby sme mali rovnaku istotu ako pri props
const navigation = useNavigation<NativeStackNavigationProp<NavigationParamList,
```

NAJPOUŽÍVANEJŠIE METÓDY

Metóda	Smer	Čo spraví
`navigate(…)`	Dopredu	Otvorí obrazovku; ak je práve zobrazená, aktualizuje jej parametre.
`push(…)`	Dopredu	Vždy pridá novú inštanciu obrazovky.
`goBack()`	Dozadu	Vráti sa o jednu obrazovku späť.
`popTo(…)`	Dozadu	Vráti sa na konkrétnu obrazovku v stacku.
`popToTop()`	Dozadu	Vráti sa na prvú obrazovku stacku.

NAVIGÁCIA V PRAXI

Dopredu: navigate vs. push

```
// stack: ProductList > ProductDetail(1)
navigation.navigate('ProductDetail', { productId: '2' });
// stack: ProductList > ProductDetail(2)  – obrazovka nepribudla, zmenili sa len
parametre

// stack: ProductList > ProductDetail(1)
navigation.push('ProductDetail', { productId: '2' });
// stack: ProductList > ProductDetail(1) > ProductDetail(2)
```

Dozadu: goBack, popTo a popToTop

```
// stack: ProductList > ProductDetail > Cart > Checkout
navigation.goBack();              // ProductList > ProductDetail > Cart
navigation.popTo('ProductList'); // ProductList
navigation.popToTop();            // ProductList
```

Počiatočné parametre – `initialParams`

Predvolené parametre obrazovky nastavíme cez `initialParams` na `Stack.Screen`.

```
<Stack.Screen
    name="ProductList"
    component={ProductListScreen}
    initialParams={{
      sortBy: 'rating',
      direction: 'asc'
    }}
/>
```

Parametre z navigácie sa s nimi **zlúčia** a prepíšu spoločné polia.

```
// initialParams: { productId: '1', showReviews: true }
navigation.navigate('ProductDetail', { productId: '42' });
// route.params === { productId: '42', showReviews: true }
```

Zmena parametrov zobrazenej obrazovky

Pri filtri alebo vyhľadávaní môže obrazovka zmeniť vlastné parametre.

- `setParams(params)` – **zlúči** s existujúcimi.
- `replaceParams(params)` – **nahradí všetky** parametre.

```
// aktuálne route.params === { productId: '42', tab: 'reviews' }

navigation.setParams({ tab: 'specs' });
// route.params === { productId: '42', tab: 'specs' }

navigation.replaceParams({ tab: 'specs' });
// route.params === { tab: 'specs' }  — productId zmizol!
```

Rezervované názvy parametrov

Tieto názvy používa React Navigation interne.
Nepoužívame ich ako vlastné parametre.

`screen` `params` `initial` `path` `state` `merge` `pop`

```
// ❌ 'screen' je rezervovany — React Navigation to vyhodnoti ako navigaciu do vnoreneho navigatora
navigation.navigate('Orders', { screen: 'detail' });

// ✅ pouzime vlastny nazov
navigation.navigate('Orders', { orderScreenId: 'detail' });
```

Čo patrí do parametrov?

Posielame **malé, serializovateľné hodnoty**.

✓ Posielame

- ID entity
  `{ productId: '42' }`
- Nastavenia obrazovky
  `{ tab: 'reviews' }`
- Krátke texty
  `{ query: 'skrutka' }`

✕ Neposielame

- Celé objekty a veľké dáta
- Funkcie a callbacky
- Inštancie tried, komponenty, blob dáta
- Heslá, tokeny, čísla kariet

Navigačný stav

Interná „mapa“ navigácie v aplikácii.

- Popisuje **obrazovky, aktívnu obrazovku, históriu** a vnorené navigátory.
- Metódy `navigate`, `push` a návraty späť **menia tento stav**.
- React Navigation podľa neho zobrazí aktívnu obrazovku.
- Využíva sa pri **deep linkingu, obnove stavu, debugovaní a analytike**.

Čo obsahuje navigačný stav?

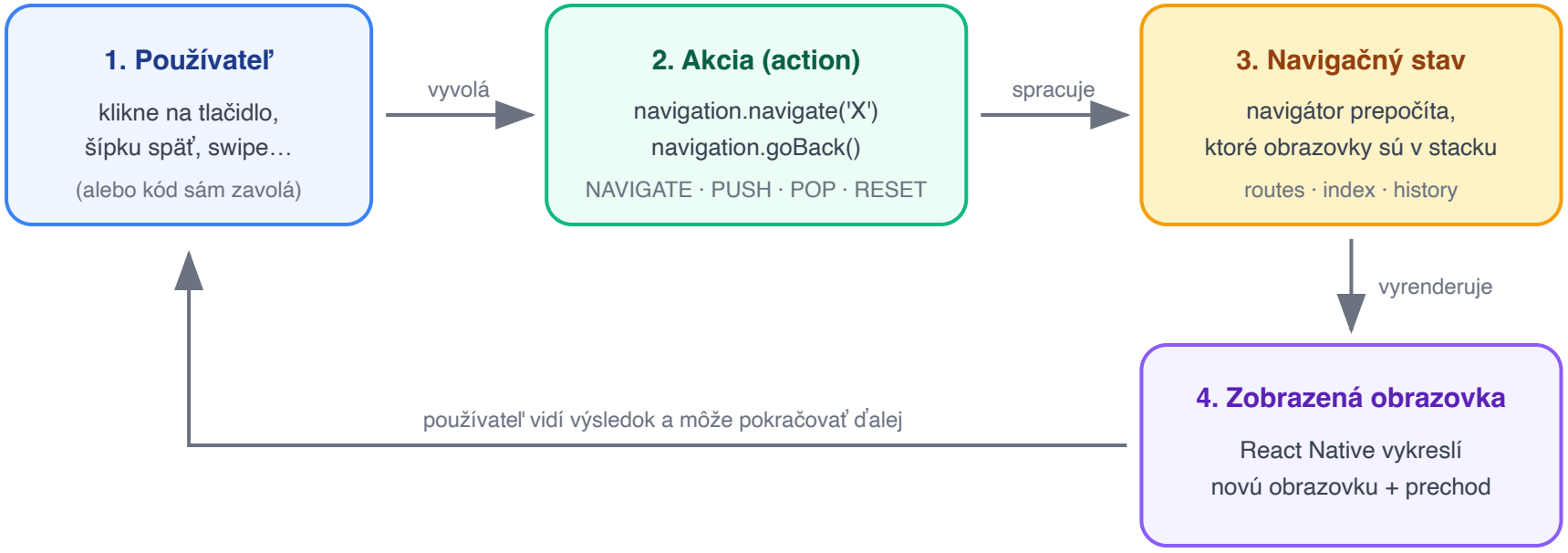
Každý navigátor má vlastný objekt stavu.

Pole	Význam
`type`	Typ navigátora: stack, tab alebo drawer.
`key`	Unikátny identifikátor vetvy stavu.
`routeNames`	Názvy obrazoviek registrovaných v navigátore.
`routes`	Objekty obrazoviek: name, key, params, vnorený state.
`index`	Index aktívnej obrazovky v poli routes.
`history`	História prechádzania, hlavne pri tab a drawer.
`stale`	Informácia o rehydratácii stavu.

PRAKTICKÁ ČASŤ

Navigačné akcie (actions)

V logoch vidíme **stav aj akciu**, ktorá ho zmenila, napríklad `NAVIGATE` alebo `POP`.



Ako vyzerá akcia?

Volanie metódy vytvorí objekt. Navigátor ho spracuje a vypočíta nový stav.

```
{
  "type": "NAVIGATE",
  "payload": { "name": "ProductDetail", "params": { "productId": "2" } },
  "source": "ProductList-xYz123",
  "target": "stack-Abc456"
}
```

- `type` – typ akcie; `payload` – údaje akcie.
- `source` – kľúč obrazovky; `target` – kľúč cieľového navigátora.

Dve skupiny akcií

`CommonActions`

Spoločné pre **stack, tab aj drawer**.

`NAVIGATE` · `GO_BACK`

`StackActions`

Špecifické pre **stack navigátor**.

`PUSH` · `POP`

Ak aktuálny navigátor akciu nespracuje, posunie ju rodičovi.

Najčastejšie akcie – `CommonActions`

Akcia	Vyvolá ju	Čo robí
`NAVIGATE`	`navigation.navigate('X')`	Prejde na X; ak je práve zobrazená, aktualizuje parametre.
`GO_BACK`	`navigation.goBack()`	Vráti sa o jednu obrazovku späť.
`SET_PARAMS`	`navigation.setParams({...})`	Zlúči parametre s existujúcimi.
`REPLACE_PARAMS`	`navigation.replaceParams({...})`	Kompletne nahradí parametre.
`RESET`	`navigation.reset({...})`	Nahradí celý navigačný stav.

Najčastejšie akcie – `StackActions`

Akcia	Vyvolá ju	Čo robí
PUSH	`navigation.push('X')`	Pridá novú inštanciu obrazovky.
POP	`navigation.pop(n)`	Odstráni n obrazoviek z vrchu stacku (predvolene 1).
POP_TO	`navigation.popTo('X')`	Vráti sa na X a zahodí obrazovky nad ňou.
POP_TO_TOP	`navigation.popToTop()`	Vráti sa na koreňovú obrazovku.
REPLACE	`navigation.replace('X')`	Nahradí aktuálnu obrazovku; história sa nerozšíri.

Priame odoslanie akcie – `dispatch`

Akciu môžeme poslať priamo cez `navigation.dispatch(...)`.

```
import { CommonActions, StackActions } from '@react-navigation/native';

// ekvivalent navigation.popToTop()
navigation.dispatch(StackActions.popToTop());

// tvrdy reset celeho stacku – v logoch uvidime akciu RESET
navigation.dispatch(
  CommonActions.reset({
    index: 0,
    routes: [{ name: 'ProductList' }],
  })
);
```

Nespracovaná akcia sa posúva k rodičovskému navigátoru.

Navigation ref – `navigationRef`

Ref na `NavigationContainer` umožňuje prístup k navigácii **zvonku React komponentov.**

```
import { NavigationContainer, useNavigationContainerRef } from '@react-
navigation/native';

export default function App() {
  const navigationRef = useNavigationContainerRef<NavigationParamList>();

  return (
    <NavigationContainer ref={navigationRef}>
      <RootStack />
    </NavigationContainer>
  );
}
```

Kedy použiť navigation ref?

Keď potrebujeme navigovať alebo čítať stav **mimo komponentu**.

- **API vrstva** – po odpovedi 401 presmerovať na Login.
- **Push notifikácie** – otvoriť konkrétnu obrazovku.
- **Analytika a logovanie** – zistiť aktuálnu obrazovku.
- **Dev nástroje** – `useLogger`, `useReactNavigationDevTools`.

Čo vieme cez ref robiť?

Navigácia

```
navigationRef.navigate('ProductDetail', { productId: '42' });
navigationRef.goBack();
navigationRef.dispatch(StackActions.popToTop());
```

Čítanie stavu

```
navigationRef.getCurrentRoute(); // aktualna obrazovka a jej parametre
navigationRef.getRootState();    // cely navigacny stav
navigationRef.canGoBack();       // true / false
```

Najprv skontrolujeme `isReady()`

Navigácia je pripravená až po namountovaní `NavigationContainer`.
Pred navigovaním cez ref overíme `isReady()`.

```
if (navigationRef.isReady()) {
  navigationRef.navigate('ProductDetail', { productId: '42' });
}
```

Vnútri obrazovky: `navigation` z props alebo `useNavigation`.

Mimo komponentov: `navigationRef` – výnimka, nie hlavný spôsob navigácie.

DEMO

Navigation lifecycle

Odchod z obrazovky **nemusí znamenať unmount**.
Pri prechode na `ProductDetail` zostáva `ProductList` namountovaný.

`focused`

Obrazovka je aktívna.
Používateľ sa na ňu pozerá.

`blurred`

Obrazovka je stále nažive,
ale nie je aktívna.

Platí aj pre taby: prepnutie tabu predchádzajúcu obrazovku nezahodí.

Eventy `focus` a `blur`

`focus` – obrazovka sa stala aktívnou. `blur` – prestala byť aktívna.

```
useEffect(() => {
  const unsubscribeFocus = navigation.addListener('focus', () => {
    console.log('ProductList je aktívny');
  });
  const unsubscribeBlur = navigation.addListener('blur', () => {
    console.log('ProductList uz nie je aktívny');
  });

  return () => {
    unsubscribeFocus();
    unsubscribeBlur();
  };
}, [navigation]);
```

Focus/blur ≠ mount/unmount. Obrazovka môže získať a stratiť focus mnohokrát.

Prečo nestačí `useEffect`?

`useEffect(..., [])` sa viaže na **mount**.
Po návrate na stále namountovanú obrazovku sa znova nespustí.

```
// nacita sa len pri namountovani
useEffect(() => {
  loadCart();
}, []);
```

- V detaile pridáme produkt do košíka.
- Vrátime sa na zoznam – počet položiek môže zostať **neaktuálny**.

`useFocusEffect`

Efekt pri **focuse**, cleanup pri **blure alebo unmounte**.

```
import { useCallback } from 'react';
import { useFocusEffect } from '@react-navigation/native';

useFocusEffect(
  useCallback(() => {
    const subscription = startLocationTracking();

    return () => {
      subscription.stop();
    };
  }, [])
);
```

Callback obalíme v `useCallback`. Použitie: refresh dát, poloha, kamera, video, subscriptions.

`useIsFocused`

Vracia `boolean` a pri zmene focusu spôsobí **re-render**.

```
import { useIsFocused } from '@react-navigation/native';

const isFocused = useIsFocused();

return isFocused ? <Camera /> : null;
```

- Použijeme na **podmienené renderovanie** kamery, mapy či videa.
- Na vedľajšie efekty, napr. fetch dát, použijeme `useFocusEffect`.

`navigation.isFocused()`

Jednorazová kontrola aktivity obrazovky.
Nespôsobuje re-render.

```
const onDataReceived = useCallback((data) => {
  if (navigation.isFocused()) {
    showToast('Prisli nove data');
  }
}, [navigation]);
```

Použijeme v event handleri, timeoute alebo po dokončení API volania – napríklad pred zobrazením upozornenia.

Focus – čo kedy použiť?

Nástroj	Typ	Re-render	Použitie
`useFocusEffect`	hook (efekt)	nie	Fetch dát pri focuse, štart/stop subscriptions.
`useIsFocused`	hook (hodnota)	áno	Podmienené renderovanie kamery, mapy, videa.
`navigation.isFocused()`	metóda	nie	Jednorazová kontrola v callbacku alebo async kóde.
`addListener('focus'/'blur')`	event listener	nie	Vlastná logika pre konkrétny event.
`useEffect`	React hook	–	Logika viazaná na mount, nie na focus.

Strata focusu nezastaví `useEffect`. Subscriptions a intervaly pokračujú, kým ich nezruší cleanup.

PRAKTICKÁ ČASŤ

Ako funguje vnorená navigácia

Navigátor je **obrazovkou iného navigátora**.

```
Drawer (RootNavigation)
├── BottomNavigation            <- celý BottomTab navigátor ako jedna "obrazovka" Draweru
│   ├── ProductsStack           <- celý Stack navigátor ako jeden "tab"
│   │   ├── ProductList
│   │   └── ProductDetail
│   └── FavoriteProductList
└── UserDetail
```

Drawer vidí `BottomNavigation` ako jednu route, hoci obsahuje ďalšie navigátory.

Stav a životný cyklus vnorených navigátorov

- **Vlastný stav a história.** Každý navigátor si ich spravuje samostatne; stavy vnorených vetiev sú v `route.state`.
- **Prvé otvorenie vetvy.** Vnorený navigátor sa namountuje, keď sa jeho route prvýkrát stane aktívnou.
- **Prepnutie tabu nie je unmount.** Obrazovka zostáva nažive, iba stratí focus. Na obnovu dát pri návrate slúži `useFocusEffect`.

Poradie navigátorov určuje UI

- **Každý navigátor má vlastné UI.** Duplicitné headre skryjeme cez `headerShown: false` na route vnoreného navigátora.
- **UI rodiča je nad dieťaťom.** Pri otvorení `ProductDetail` v `ProductsStack` zostane tab bar viditeľný.
- **Poradie určuje výsledok.** Tab nad stackom zachová tab bar; stack nad tabmi umožní detailu prekryť ho.
- **Fullscreen modal patrí hore.** Aby prekryl aj UI rodiča, umiestnime ho do koreňového navigátora.

Metódy rodičov sa dedia nadol

V `ProductList` máme aj metódy nadradených navigátorov.

Navigátor	Príklady metód
Stack	`push`, `popTo`
Tab	`jumpTo`
Drawer	`openDrawer`, `closeDrawer`, `toggleDrawer`

Dostupné metódy závisia od hierarchie navigátorov.

Eventy rodiča sa nededia

Obrazovka vnoreného navigátora automaticky nedostane `tabPress` ani `drawerItemPress`.

```
useEffect(() => {
  const unsubscribe = navigation.getParent()?.addListener('tabPress', (e) => {
    // napr. scroll na zaciatok zoznamu pri opatovnom kliknuti na tab
  });
  return unsubscribe;
}, [navigation]);
```

Na event rodiča sa prihlásime cez `navigation.getParent()`.

Navigácia do inej vetvy

Uvedieme **cestu**: route rodiča → `screen` → `params`.

```
navigation.navigate('ProductsStack', {
  screen: 'ProductDetail',
  params: { productId: '42' },
});
```

```
navigation.navigate('BottomNavigation', {
  screen: 'ProductsStack',
  params: {
    screen: 'ProductDetail',
    params: { productId: '42' },
  },
});
```

Prvé otvorenie vetvy – `initial: false`

Pri prvom otvorení detailu chceme pod ním aj `ProductList`, aby fungovalo tlačidlo späť.

```
navigation.navigate('ProductsStack', {
  screen: 'ProductDetail',
  params: { productId: '42' },
  initial: false, // stack: ProductList > ProductDetail
});
```

```
<Stack.Navigator initialRouteName="ProductList">
  <Stack.Screen name="ProductList" component={ProductListScreen} />
  <Stack.Screen name="ProductDetail" component={ProductDetailScreen} />
</Stack.Navigator>
```

Bez `initial: false` môže byť cieľový detail jedinou obrazovkou nového stacku.

Ako putujú akcie (bubbling)

1. Akciu spracuje **najbližší navigátor**, ktorý ju vie spracovať.
2. Ak ju spracovať nevie, akcia sa **posunie rodičovi** (bubbling nahor).
3. Ak ju nevie spracovať nikto, nestane sa nič.

Preto:

- `goBack()` z `ProductDetail` nezavrie tab, ale vráti nás na `ProductList` – akciu spracuje stack, ktorý má kam ísť späť.
- `goBack()` na `ProductList` (prvá obrazovka stacku) sa posunie vyššie – späť ho spracuje tab/drawer podľa vlastnej histórie.
- `openDrawer()` v `ProductList` funguje, hoci `ProductList` je v stacku – akcia prebubláva cez tab až k Draweru.
- `popToTop()` v tab navigátore nespraví nič, ak nad ním nie je žiadny stack.

Cielené doručenie akcie vieme vynútiť cez `target`, teda `key` konkrétneho navigátora (`navigation.dispatch({ ...StackActions.popToTop(), target: stackKey })`). V praxi to potrebujeme len výnimočne.

Typovanie – `CompositeScreenProps`

Spojí typy **obrazovky a jej rodiča**, aby TypeScript poznal aj navigáciu do iných vetiev.

```
export type ProductStackScreenProps<T extends keyof ProductStackParamList> =
  CompositeScreenProps<
    NativeStackScreenProps<ProductStackParamList, T>, // vlastny navigator
    BottomTabNavigationScreenProps<'ProductsStack'> // rodic
  >;
```

- Reťazíme **smerom nahor**: screen → tab → drawer.
- `NavigatorScreenParams<ChildParamList>` typuje vnorené `{ screen, params }`.
- Pre samotný objekt z `useNavigation` použijeme `CompositeNavigationProp`.

Akcie pre taby – `TabActions`

`JUMP_TO` prepne na existujúci tab a môže mu odovzdať parametre.

```
import { TabActions } from '@react-navigation/native';

navigation.dispatch(TabActions.jumpTo('FavoriteProductList'));
```

- Metóda: `navigation.jumpTo('X', params)`.
- `jumpTo` funguje iba v rámci tabov.
- `navigate` je univerzálny a môže akciu posunúť rodičovi.

Akcie pre drawer – `DrawerActions`

Akcia	Metóda	Výsledok
`OPEN_DRAWER`	`openDrawer()`	Otvorí menu.
`CLOSE_DRAWER`	`closeDrawer()`	Zatvorí menu.
`TOGGLE_DRAWER`	`toggleDrawer()`	Prepne otvorené/zatvorené.
`JUMP_TO`	`jumpTo('X', params)`	Prepne na obrazovku X.

```
import { DrawerActions } from '@react-navigation/native';

navigation.dispatch(DrawerActions.openDrawer());
navigation.dispatch(DrawerActions.jumpTo('UserDetail'));
```

Časté chyby – UI a eventy

Problém	Príčina	Riešenie
Viacero headrov nad sebou	každý navigátor renderuje vlastný header	`options={{ headerShown: false }}` na route vnoreného navigátora
Tab bar / drawer UI zostáva viditeľné na detaile	UI rodiča sa renderuje **nad** dieťaťom	prehodiť poradie navigátorov (stack nad tab), alebo skryť tab bar cez `options`
Modal neprekryje celú obrazovku	modal je v hlbšie vnorenom navigátore	presunúť modal do koreňového navigátora
Obrazovka nereaguje na `tabPress`	eventy rodiča sa do dieťaťa nešíria	`navigation.getParent()?.addListener('tabPress', ...)`

Časté chyby – história a typy

Problém	Príčina	Riešenie
Chýbajúce back tlačidlo po navigácii do vnoreného stacku	stack sa mountuje priamo na cieľovej obrazovke	`initial: false`
Back tlačidlo ma nevráti tam, kam čakám	akciu spracoval iný navigátor (bubbling)	uvedomiť si hierarchiu, prípadne použiť `popTo` / `target`
TS chyba pri navigácii do inej vetvy	screen props pozná len vlastný navigátor	`CompositeScreenProps` / `CompositeNavigationProp`

Časté chyby – lifecycle a metódy

Problém	Príčina	Riešenie
Dáta sa po návrate neobnovia	obrazovka zostáva namountovaná	`useFocusEffect` namiesto `useEffect`
Efekt sa spustí aj bez focusu	namountovaná obrazovka reaguje cez `useEffect` na zmenu závislostí aj na pozadí	logiku potrebnú iba pri focuse presunúť do `useFocusEffect`, subscriptions rušiť v cleanupe
Neaktívna obrazovka sa zbytočne prerenderováva	stále odoberá zmeny zdieľaného stavu alebo jej efekt aktualizuje lokálny stav	zúžiť odber stavu cez selektory a aktualizácie potrebné iba pri focuse viazať na `useFocusEffect`
`jumpTo` hodí chybu	`jumpTo` existuje len v tab/drawer navigátore	použiť `navigate`

Nakreslime si strom navigátorov

Navigátor je len ďalšia obrazovka.

```
Drawer (RootNavigation)
├── BottomNavigation
│   ├── ProductsStack
│   │   ├── ProductList
│   │   └── ProductDetail
│   └── FavoriteProductList
└── UserDetail
```

Zo stromu vieme odvodiť cestu k obrazovke, dostupné metódy aj to, ktorý navigátor spracuje akciu.

PRAKTICKÁ ČASŤ

Čo je deep link?

URL, ktorá otvorí aplikáciu priamo na konkrétnej obrazovke.

```
partsformachine://product-detail/42
```

- Kliknutie na odkaz → priamo detail produktu.
- Push notifikácie, e-maily, QR kódy a zdieľanie obsahu.
- Marketingové kampane a prechod z webu do aplikácie.

Dva typy odkazov

Porovnanie	Custom scheme	Universal Links / App Links
URL	`partsformachine://product-detail/42`	`https://partsformachine.sk/product-detail/42`
Nastavenie	Registrácia schémy v aplikácii.	Aj overenie vlastníctva domény.
Bez aplikácie	Odkaz nefunguje.	Otvorí sa web s rovnakým obsahom.
Bezpečnosť	Schému môže zaregistrovať aj iná appka.	Odkaz je viazaný na overenú doménu.
Použitie	Interné odkazy, vývoj, testovanie.	Verejné produkčné odkazy.

Ako URL spracuje React Navigation

Podľa konfigurácie `linking` preloží URL na **navigačný stav**.

```
URL
 ↓
linking.config
 ↓
navigation state
 ↓
zobrazená obrazovka
```

Konfigurácia odkazov

- `prefixes` – URL prefixy aplikácie.
- `config.screens` – mapa obrazoviek na cesty v URL.

```
const linking = {
  prefixes: [Linking.createURL('/'), 'https://partsformachine.sk'],
  config: {
    screens: {
      ProductDetail: 'product-detail/:productId',
    },
  },
};
```

Vnorené `screens` kopírujú strom navigátorov. Bez `config` sa URL mapuje podľa názvov obrazoviek.

Užitočné možnosti konfigurácie

Možnosť	Načo slúži
`:paramName`	Povinný parameter z cesty.
`:paramName?`	Voliteľný parameter.
`parse` / `stringify`	Konverzia hodnôt parametrov.
`initialRouteName`	Počiatočná obrazovka pod cieľovou pre návrat späť.
`exact: true`	Vnorená obrazovka nededí cestu rodiča.
`path: '*'`	Fallback obrazovka pre neznáme URL.

Príklad konverzie parametra – `parse`

Hodnotu z URL upravíme pred odovzdaním obrazovke.

```
ProductDetail: {
  path: 'product-detail/:productId',
  parse: { productId: (id: string) => id.toUpperCase() },
},
```

`product-detail/b3m8q1` → `route.params.productId === 'B3M8Q1'`

Testovanie deep linkov

```
# iOS simulator
xcrun simctl openurl booted "partsformachine://product-detail/b3m8q1"

# Android emulator
adb shell am start -W -a android.intent.action.VIEW -d "partsformachine://product-
detail/b3m8q1"
```

V Expo Go má URL iný prefix:

```
exp://192.168.4.114:8081/--/product-detail/b3m8q1
```

`Linking.createURL('/')` vygeneruje správny prefix pre Expo Go aj build.

Deep linking – dokumentácia

Prebrali sme iba úvod do deep linkingu.
Viac informácií nájdete v dokumentácii:

reactnavigation.org/docs/deep-linking

reactnavigation.org/docs/configuring-links

PRAKTICKÁ ČASŤ

Podmienené renderovanie obrazoviek

Stav určuje, ktoré obrazovky existujú.

```
zmena stavu (isLoggedIn)
          ↓
iná sada obrazoviek
          ↓
prekreslenie navigátora
```

Po prihlásení nemusíme volať `navigate('UserDetail')`. Navigáciu riadi stav.

Dva prístupy k navigácii

Porovnanie	Imperatívny prístup	Podmienené renderovanie
Kto rozhoduje	Obrazovka volá `navigate()`.	Stav určuje, čo je registrované.
Login v histórii	Zostáva; treba ho odstrániť cez `reset`.	Zanikne automaticky.
Riziko	Návrat späť do chránenej časti.	Na neexistujúcu route sa nedá navigovať.

Dôsledky podmieneného renderovania

- **Bez `navigate()` a `reset()`.** Po `setIsLoggedIn(true)` sa navigátor prekreslí sám.
- **Bez návratu na odregistrovanú obrazovku.** Odmountuje sa a jej stav sa zahodí.
- **Ani deep link neotvorí odregistrovanú route.**
- V logoch sa môže zobraziť `@@UNKNOWN` – stav sa zmenil bez dispatchnutej akcie.

Čo ak zanikne aktívna route?

- **Navigátor zvolí prežívajúcu obrazovku.** V našom príklade sa focus vráti na `ProductList`.
- **Ak má používateľ zostať na mieste:** obom vetvám dáme rovnaký `name`. Route zostane rovnaká, vymení sa jej obsah.

Viac o prihlasovaní

V reálnej aplikácii by sme pridali aj **loading state**.

Viac informácií nájdete v dokumentácii:

reactnavigation.org/docs/auth-flow

PRAKTICKÁ ČASŤ

PRAKTICKÁ ČASŤ

Čo je modál?

Modál je obrazovka, ktorá sa nezobrazí „vedľa“ ostatných v rovnakej "vrstve", ale **navrchu** – prekryje aktuálny obsah a používateľ ju musí zavrieť, aby mohol pokračovať. Typicky prichádza **zdola nahor** a predchádzajúca obrazovka ostáva čiastočne viditeľná v pozadí.

Kedy sa modál hodí:

- **prerušenie hlavného toku** – filter, výber položky, potvrdenie akcie,
- **krátka, ukončená úloha** – prihlásenie, formulár, sprievodca,
- **obsah, ktorý nepatrí do histórie** – používateľ ho zavrie a vráti sa presne tam, odkiaľ prišiel.

Modál v React Navigation **nie je samostatný komponent** (ako `Modal` z React Native). Je to **bežná obrazovka stacku**, ktorá má len iný spôsob zobrazenia – nastavuje sa cez `options={{ presentation: 'modal' }}`.

Ako sa obrazovka zobrazí – `presentation`

Prop `presentation` určuje **spôsob prezentácie** obrazovky v stacku:

Hodnota	Čo robí
`card`	predvolené správanie – obrazovka sa zasunie zboku ako bežná karta v stacku
`modal`	obrazovka príde **zdola nahor**, pod ňou vidno zmenšenú predchádzajúcu obrazovku (iOS štýl)
`transparentModal`	pozadie obrazovky je **priehľadné** – predchádzajúca obrazovka ostáva plne viditeľná (vhodné pre vlastné dialógy, bottom sheety)

Rozdiel medzi platformami: na **iOS** má `modal` charakteristickú „kartičkovú“ prezentáciu so zaoblenými rohmi a gestom potiahnutia nadol. Na **Androide** natívny systém takúto prezentáciu nemá, preto výsledok nemusí vyzerať rovnako.

Modál v koreňovom navigátore

Platí pravidlo z kapitoly o vnorenej navigácii: **UI rodiča sa renderuje nad UI dieťaťa**. Ak je modál vnútri hlboko vnoreného stacku, **neprekryje** tab bar ani header rodiča a nebude naozaj „fullscreen“.

Animácie prechodov

Každý prechod medzi obrazovkami je **animácia**, ktorú vieme ovplyvniť cez `options` na `Stack.Screen` alebo na `Stack.Navigator` (vtedy platí pre všetky obrazovky).

Predvolené správanie

Bez akéhokoľvek nastavenia sa React Navigation správa **podľa platformy** – na iOS sa obrazovka zasunie sprava, na Androide použije animáciu blízku systémovej. Cieľom je, aby appka pôsobila natívne bez toho, aby sme čokoľvek nastavovali.

Prednastavené animácie – `animation`

Najjednoduchší spôsob, ako animáciu zmeniť, je prop `animation`. Namiesto popisovania celej animácie len vyberieme **jednu z prednastavených**:

Hodnota	Ako prechod vyzerá
`default`	animácia podľa platformy (predvolené)
`slide_from_right`	obrazovka sa zasunie sprava (typické pre iOS)
`slide_from_left`	obrazovka sa zasunie zľava
`slide_from_bottom`	obrazovka príde zdola – typické pre modály
`fade`	obrazovka sa jednoducho prelína (fade in/out)

Ďalšie animácie – `animation`

Hodnota	Ako prechod vyzerá
`fade_from_bottom`	obrazovka sa prelína a zároveň mierne prichádza zdola (Android štýl)
`reveal_from_bottom`	obrazovka sa „odhalí“ zdola, stará ostáva na mieste (starší Android štýl)
`scale_from_center`	obrazovka sa zväčší zo stredu (Android 10+ štýl)
`none`	žiadna animácia – obrazovka sa vymení okamžite

Pri prepnutí na iný typ animácie sa zvyčajne prispôsobí aj **gesto** – napr. pri `slide_from_bottom` sa obrazovka zatvára potiahnutím nadol, nie doprava.

Hotové prechody – `TransitionPresets`

Okrem jednoduchých hodnôt ponúka `@react-navigation/stack` aj **hotové presety**, ktoré obsahujú kompletné nastavenie prechodu (animácia obrazovky, animácia headera aj gesto). Sú to napríklad prechody v štýle iOS kariet, iOS modálov alebo rôznych verzií Androidu.

Preset	Ako prechod vyzerá
`DefaultTransition`	predvolený prechod podľa platformy
`SlideFromRightIOS`	klasický iOS prechod – obrazovka sa zasunie sprava
`ModalSlideFromBottomIOS`	modál, ktorý príde zdola cez celú obrazovku
`ModalPresentationIOS`	iOS „kartičkový“ modál – predchádzajúca obrazovka sa zmenší a odsunie dozadu

Ďalšie prechodové presety

Preset	Ako prechod vyzerá
`FadeFromBottomAndroid`	Android štýl – prelínanie s miernym posunom zdola
`RevealFromBottomAndroid`	Android štýl – obrazovka sa „odhalí“ zdola
`ScaleFromCenterAndroid`	Android 10+ štýl – obrazovka sa zväčší zo stredu
`BottomSheetAndroid`	prechod v štýle spodného panela (bottom sheet)

Použitie prechodového presetu

Preset stačí „rozbaliť“ do `options` – buď pre celý navigátor, alebo pre jednu obrazovku:

```
import { TransitionPresets } from '@react-navigation/stack';

// pre vsetky obrazovky navigatora
<RootStack.Navigator
  screenOptions={{ ...TransitionPresets.SlideFromRightIOS }}
>
  {/* len pre jednu obrazovku */}
  <RootStack.Screen
    name="ProductFilter"
    component={ProductFilterScreen}
    options={{ ...TransitionPresets.ModalPresentationIOS }}
  />
</RootStack.Navigator>
```

Prepísanie hodnôt presetu

Preset vieme aj **prepísať** – stačí zaň dopísať vlastné hodnoty:

```
options={{
...TransitionPresets.ModalSlideFromBottomIOS,
gestureEnabled: false,   // modal sa nebude dat zavriet gestom
}}
```

Preset je len „balíček“ hodnôt, ktoré by sme inak nastavovali ručne. Preto je najrýchlejšou cestou, ako dosiahnuť konzistentný natívny vzhľad naprieč celou aplikáciou.

Jemné doladenie animácie

Ak nám preset nestačí, `@react-navigation/stack` umožňuje animáciu **rozobrať na časti**:

Nastavenie	Čo ovplyvňuje
`transitionSpec`	**časovanie** animácie – zvlášť pre otvorenie (`open`) a zvlášť pre zatvorenie (`close`); vieme zvoliť `timing` (pevné trvanie) alebo `spring` (pružinové správanie)
`cardStyleInterpolator`	**ako sa obrazovka vizuálne mení** počas prechodu – posun, priehľadnosť, zväčšenie, tieň
`headerStyleInterpolator`	rovnaká logika, ale pre **header** – aby sa titulok a tlačidlá animovali spolu s obsahom
`gestureEnabled`	či sa dá obrazovka zavrieť **gestom** (potiahnutím)
`gestureDirection`	**smer gesta** – vodorovný pri kartách (`horizontal`), zvislý pri modáloch (`vertical`)

Na čo si dať pozor

- **Nepreháňať to.** Animácia má prechod uľahčiť, nie naň upozorňovať. Dlhé alebo neobvyklé prechody pôsobia pomaly.
- **Rešpektovať platformu.** Používatelia iOS a Androidu sú zvyknutí na iné správanie – vlastnú animáciu nasadzujeme len vtedy, keď na to máme dôvod.
- **Nezabudnúť na gesto.** Ak vypneme animáciu alebo zmeníme smer prechodu, musí tomu zodpovedať aj gesto na zavretie obrazovky.
- **Myslieť na prístupnosť.** Systém má nastavenie „obmedziť pohyb“ (reduce motion) – aplikácia by ho mala rešpektovať.
- **Testovať na oboch platformách.** Rovnaké nastavenie môže na iOS a Androide vyzerať odlišne.

Stack vs. native stack

Dve knižnice ukladajú obrazovky na seba, ale **vnútorne fungujú inak**.

- ** `@react-navigation/stack` – **celý stack je „poskladaný“ v** JavaScripte**. Obrazovky sú bežné React Native `View`-čka a prechody aj gestá si knižnica animuje sama (cez `Animated` a gesture handler).
- ** `@react-navigation/native-stack` – **knižnica používa** natívne navigátory platformy** (`UINavigationController` na iOS, `Fragment` na Androide). Prechody, gestá aj header vykresľuje priamo systém.

Porovnanie – výkon a vzhľad

Porovnanie	`stack`	`native-stack`
Kto vykonáva animáciu	JavaScript (`Animated` + gesture handler)	**natívny** navigátor platformy
Výkon	veľmi dobrý, ale závisí od JS vrstvy	**najlepší**, animácia beží mimo JS
Vzhľad a pocit	„namaľovaný“ knižnicou – rovnaký na oboch platformách	presne taký, aký má daná platforma
Header	React komponent, dá sa ľubovoľne štýlovať	natívny header – štýlovanie je obmedzené

Porovnanie – úpravy a závislosti

Porovnanie	`stack`	`native-stack`
Možnosti úprav animácie	**veľké** – vlastné časovanie aj interpolátory	**obmedzené** – vyberáme z toho, čo platforma ponúka
Podpora animácií	konzistentná na iOS aj Androide	závisí od platformy a jej verzie – niektoré varianty sa môžu líšiť
Gestá	plne konfigurovateľné (`gestureDirection`, `gestureResponseDistance`)	natívne gesto platformy, menej nastavení
Veľkosť a závislosti	potrebuje `react-native-gesture-handler` (a `masked-view` pre niektoré prechody)	stavia na `react-native-screens`

Kedy zvoliť `native-stack`?

Predvolená voľba pre bežnú navigáciu.

- chceš **maximálny výkon** a natívny pocit aplikácie,
- ti stačí štandardný vzhľad headera aj prechodov,
- appka má veľa obrazoviek a záleží ti na plynulosti.

Kedy zvoliť `stack`?

- potrebuješ **vlastnú animáciu prechodu** (`transitionSpec`, `cardStyleInterpolator`),
- chceš **rovnaký vzhľad na iOS aj Androide**,
- potrebuješ **plne štýlovateľný header** alebo špecifické gestá,
- štandardná prezentácia (napr. modál na Androide) nevyzerá tak, ako potrebuješ.

Obidva navigátory sa dajú v jednej aplikácii **kombinovať** – väčšinu appky postavíme na `native-stack` a klasický `stack` použijeme len tam, kde naozaj potrebujeme vlastné správanie.

PRAKTICKÁ ČASŤ

Čo je safe area?

Časť obrazovky, ktorú **neprekrýva hardvér ani systémové UI**.

- Notch, Dynamic Island, status bar a zaoblené rohy.
- Home indikátor a navigačný panel Androidu.
- Mimo safe area môže byť obsah **prekrytý alebo orezaný**.

Insets – dynamické odsadenia

Hodnoty `top`, `bottom`, `left` a `right` určujú bezpečné odsadenie.

- **Menia sa** podľa zariadenia, orientácie a navigačného UI.
- React Navigation odsadí **vlastný header a tab bar**.
- O **obsah obrazovky** sa staráme sami – najmä pri `headerShown: false` alebo vlastnom headri.

Prečo nestačí pôvodný `SafeAreaView`?

Komponent z `react-native` má obmedzenia:

- funguje **len na iOS 11+**, na Androide neurobí nič,
- **nevie o navigácii** – nepozná header ani tab bar, takže odsadenie môže byť zdvojené alebo chýbajúce.

Použijeme knižnicu ** `react-native-safe-area-context` **.

Nástroje pre safe area

Z knižnice `react-native-safe-area-context`:

Nástroj	Čo robí	Kedy použiť
`SafeAreaProvider`	poskytuje hodnoty insets celej aplikácii	musí obaľovať appku (v Expo a `NavigationContainer` býva už zabezpečený)
`SafeAreaView`	`View`, ktorá si sama pridá padding podľa insets	jednoduché obrazovky, kde stačí odsadiť celý obsah
`useSafeAreaInsets()`	vráti čísla `top`, `bottom`, `left`, `right`	keď potrebujeme **kontrolu**, kam odsadenie pôjde

```
const insets = useSafeAreaInsets();
<View style={{ paddingTop: insets.top }} />
```

Safe area – na čo si dať pozor

- **Insets nikdy nehardcodujeme.** Výška notchu ani home indikátora nie je na všetkých zariadeniach rovnaká.
- **Pozor na scroll view a listy.** Padding patrí do `contentContainerStyle`, nie na samotný `ScrollView` / `FlatList` – inak sa oreže obsah pri scrollovaní.
- ** `edges` obmedzujú strany.** Ak obrazovka už má header, hornú stranu netreba odsadzovať: `<SafeAreaView edges={['bottom']}>`.
- **Obsah „pod“ notchom.** Ak má obrázok alebo pozadie ísť až pod status bar, `SafeAreaView` použiť nevieme – vtedy siahneme po `useSafeAreaInsets` a odsadíme len text či tlačidlá.
- **Testovať na oboch platformách** a aj v **landscape** orientácii, kde sú dôležité aj `left` a `right` insets.

PRAKTICKÁ ČASŤ

Čo je téma?

Objekt s **farbami a fontami** pre UI navigácie.
Všetky hodnoty máme **na jednom mieste**.

```
import { NavigationContainer, DefaultTheme } from '@react-navigation/native';

const MyTheme = {
  ...DefaultTheme,
  colors: {
    ...DefaultTheme.colors,
    background: 'rgb(91, 196, 191)',
    primary: 'rgb(193, 66, 67)',
  },
};
```

Tému odovzdáme cez prop `theme` do `NavigationContainer`:

```
<NavigationContainer theme={MyTheme}>
  <RootNavigation />
</NavigationContainer>
```

Čo téma obsahuje

Kľúč	Kde sa prejaví
`colors.primary`	aktívne prvky – tlačidlá v headeri, aktívny tab, back button
`colors.background`	pozadie obrazoviek
`colors.card`	pozadie headera, tab baru a drawera
`colors.text`	text v headeri a v navigačných komponentoch
`colors.border`	oddeľovacie čiary (napr. pod headerom)
`colors.notification`	badge v tab bare
`fonts`	fonty použité v headeri a navigačných komponentoch

Téma vo vlastných komponentoch

Rovnaké hodnoty čítame cez `useTheme()`.
Farby nemusíme duplikovať v štýloch.

```
import { useTheme } from '@react-navigation/native';

const { colors } = useTheme();
<Text style={{ color: colors.text }}>Produkty</Text>
```

PRAKTICKÁ ČASŤ

Kde sa header nastavuje

- `options` – nastavenia jednej obrazovky.
- `screenOptions` – nastavenia všetkých obrazoviek navigátora.
- `options` môže byť aj funkcia s prístupom k `route` a `navigation`.

```
<Stack.Navigator
  screenOptions={{ headerBackButtonDisplayMode: 'minimal' }}
>
  <Stack.Screen
    name="ProductList"
    options={{ title: 'Produkty' }}
    component={ProductListScreen}
  />
</Stack.Navigator>
```

Zmena headera za behu – `setOptions`

Keď hodnotu poznáme až za behu, header zmeníme **priamo z obrazovky**.

```
useEffect(() => {
  navigation.setOptions({ title: product.name });
}, [navigation, product]);
```

Napríklad názov produktu načítaný z API alebo tlačidlo závislé od stavu formulára.

Header – obsah a viditeľnosť

Option	Čo robí
`title`	text nadpisu (prázdny `''` header skryje text, ale header ostane)
`headerShown`	zapne/vypne celý header
`headerLeft` / `headerRight`	vlastný komponent vľavo/vpravo (funkcia vracajúca JSX)
`headerTitle`	vlastný komponent namiesto textového nadpisu (napr. logo)
`headerStyle`	štýl pozadia headera (`backgroundColor`)

Header – vzhľad a back tlačidlo

Option	Čo robí
`headerTintColor`	farba textu a tlačidiel v headeri
`headerTitleStyle`	štýl nadpisu (font, veľkosť, hrúbka)
`headerBackButtonDisplayMode`	ako sa zobrazí text pri back tlačidle (`default` , `generic` , `minimal`) – native stack
`headerTransparent`	header sa vykreslí nad obsahom bez pozadia

Pozor: `headerLeft` na iOS **nahrádza back tlačidlo**. Ak ho chceme zachovať, musíme si ho vykresliť sami (alebo použiť `headerBackVisible`).

Tlačidlá v headeri

`headerRight` a `headerLeft` sú **funkcie vracajúce komponent.**

```
<Stack.Screen
  name="ProductList"
  options={{
    title: 'Produkty',
    headerLeft: () => <DrawerMenuButton />,
  }}
  component={ProductListScreen}
/>
```

Ak tlačidlo potrebuje stav alebo dáta obrazovky, nastavíme ho cez `navigation.setOptions`.

Vlastný header

Ak `options` nestačia, cez `header` nahradíme **celý header**.

```
<Stack.Screen
  name="ProductList"
  options={{
    header: () => <MyCustomHeader />,
  }}
  component={ProductListScreen}
/>
```

- **Safe area riešime sami** cez `useSafeAreaInsets`.
- Alternatíva: `headerShown: false` a vlastný header priamo v obrazovke.

PRAKTICKÁ ČASŤ

Kedy zabrániť návratu?

Niekedy je odchod z obrazovky **deštruktívny** – používateľ má rozpísaný formulár, prebieha nahrávanie súboru alebo neuložená objednávka. Ak by sme ho nechali len tak odísť, dáta ticho zahodíme. Preto odchod **zachytíme** a necháme používateľa potvrdiť, či naozaj chce odísť.

Blokovanie odchodu – `usePreventRemove`

Hook `usePreventRemove` má dva argumenty – **podmienku** (kedy blokovať) a **callback**, ktorý sa zavolá pri pokuse o odchod:

```
usePreventRemove(hasUnsavedChanges, ({ data }) => {
  // data.action = povodna navigacna akcia, ktoru sme zablokovali
  navigation.dispatch(data.action); // pustime pouzivatela prec
});
```

- ak je podmienka `false`, hook nerobí nič a navigácia funguje normálne,
- ak je `true`, odchod sa **zruší** a spustí sa callback,
- ak v callbacku **nič nespravíme**, používateľ zostane na obrazovke,
- ak zavoláme `navigation.dispatch(data.action)`, pôvodná akcia sa vykoná.

Čo zachytí `usePreventRemove`?

Spôsob odchodu	Zachytené
back tlačidlo v headeri	áno
hardvérové/gestové back na Androide	áno
swipe gesto späť na iOS	áno
`navigation.goBack()`, `pop()`, `popToTop()`	áno
odstránenie obrazovky podmieneným renderovaním (napr. logout)	**nie**

`usePreventRemove` je novší a odporúčaný spôsob – nahrádza manuálne počúvanie eventu `beforeRemove` cez `navigation.addListener`.

Vypnutie ciest späť (`gestureEnabled: false`, `headerBackVisible: false`) **nie je náhrada** – iba skryje tlačidlo a gesto, ale dáta nechráni a používateľa zbytočne uväzní.

Statická vs. dynamická navigácia

React Navigation ponúka **dva spôsoby**, ako zadefinovať strom obrazoviek. Oba robia to isté – rozdiel je len v tom, **ako navigáciu zapíšeme**. Runtime správanie (`navigation.navigate`, akcie, navigačný stav) je rovnaké.

Dynamické API

Navigátor a obrazovky sú **React komponenty**, ktoré renderujeme:

```
<Stack.Navigator>
  <Stack.Screen name="ProductList" component={ProductListScreen} />
  {isLoggedIn && <Stack.Screen name="UserDetail" component={UserDetailScreen} />}
</Stack.Navigator>
```

Vďaka tomu vieme obrazovky **podmienene renderovať** podľa stavu aplikácie (presne tak, ako sme to robili pri prihlásení používateľa). Typy si však píšeme ručne – `NavigationParamList` + `NativeStackScreenProps`.

Statické API

Obrazovky zapíšeme ako **konfiguračný objekt** a navigáciu vytvoríme cez `createStaticNavigation`:

```
const RootStack = createNativeStackNavigator({
  screens: {
    ProductList: {
      screen: ProductListScreen,
      options: { title: 'Produkty' },
    },
    ProductDetail: ProductDetailScreen,
  },
});

const Navigation = createStaticNavigation(RootStack);

export default function App() {
  return <Navigation />;
}
```

Statické API – odvodené typy

Typy sa dajú **odvodiť z konfigurácie** (`StaticParamList`), takže ich nemusíme písať duplicitne:

```
type RootStackParamList = StaticParamList<typeof RootStack>;
```

Statické API – podmienené obrazovky

Podmienené obrazovky sa riešia cez `if` v konfigurácii spolu s hookmi `useIsSignedIn` / `useIsSignedOut`:

```
const RootStack = createNativeStackNavigator({
  screens: {
    ProductList: ProductListScreen,
  },
  groups: {
    SignedIn: {
      if: useIsSignedIn,
      screens: { UserDetail: UserDetailScreen },
    },
    SignedOut: {
      if: useIsSignedOut,
      screens: { Login: LoginScreen },
    },
  },
});
```

Porovnanie – zápis a typy

Porovnanie	Dynamické API	Statické API
Zápis obrazoviek	JSX komponenty `<Stack.Screen>`	konfiguračný objekt `screens: {...}`
Typovanie	ručne písaný param list	odvodené cez `StaticParamList`
Podmienené obrazovky	podmienené renderovanie (`isLoggedIn && ...`)	`if` + `useIsSignedIn` / `useIsSignedOut`

Porovnanie – linking a flexibilita

Porovnanie	Dynamické API	Statické API
Deep linking	ručná `linking.config` s cestami	vygeneruje sa z konfigurácie (stačí `linking.prefixes`)
Prístup k contextu/hookom pri definícii	áno, priamo v komponente	len cez hooky v `if` a `useLayout`-style API
Flexibilita za behu	vyššia (plná kontrola v JSX)	nižšia (konfigurácia je statická)

- statické API je dostupné od **React Navigation v7**,
- obe API sa dajú **kombinovať** – napr. statický root a dynamický vnorený navigátor (a naopak),